

ส่วนที่ 1 — System Design

1.1 Deterministic Simulation

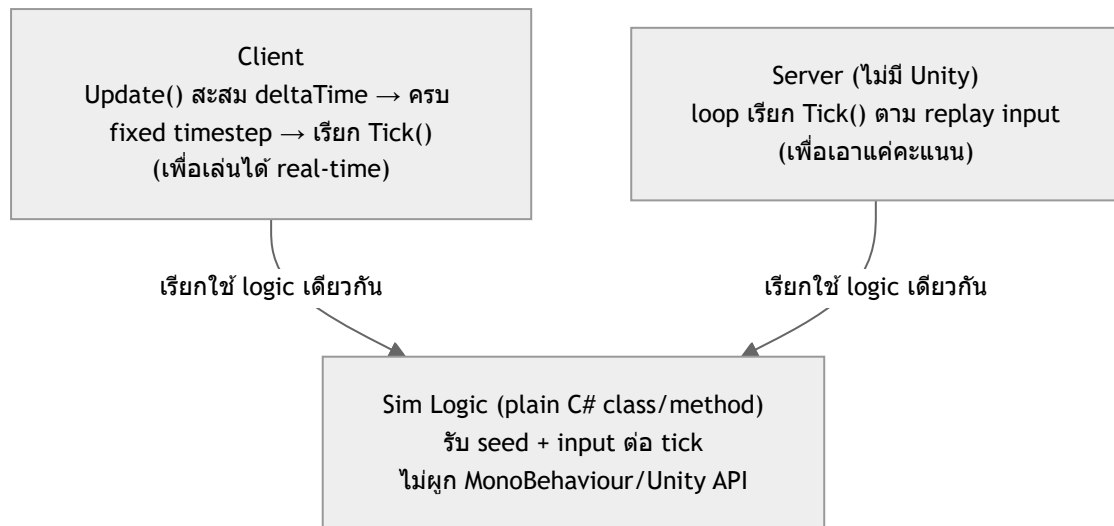
- ทำให้เกม Deterministic ได้อย่างไรบน Unity : ส่วนแสดงผลจะใช้ fixed timestep ร่วมกับ fixedupdate เพื่อให้แสดงผลเหมือนกัน และยังสามารถ simulate ออกมาใหม่ได้ เพราะเราไม่ได้ผูกกับ เวลา เราผูกกับ tick และ random สิ่งกีดขวางนั้น ต้องมี seed ที่เหมือนกัน สำหรับทุกคน เพื่อให้สิ่งกีดขวางขึ้นมาเหมือนเดิมทุกครั้งที่เล่น
- ปัจจัยที่ทำลาย Determinism + วิธีจัดการแต่ละปัจจัย :

ปัจจัยที่ทำลาย Determinism	วิธีจัดการ
การ random โดยไม่ใช้ seed ทุกครั้งที่เล่นสิ่งกีดขวางจะไม่เหมือนกัน	random โดยใช้ seed
การรันทุกอย่างใน update โดยอิง time.deltatime สุดท้ายแล้วมันจะมีความคลาดเคลื่อนเกิดขึ้น ในเครื่องที่ framerate ไม่เท่ากัน	ใช้ fixed update
แต่ละฮาร์ดแวร์มีการคำนวณค่า float ที่ต่างกันทำให้ค่า score สุดท้ายมีโอกาสคลาดเคลื่อน	round float จากการคำนวณทุก tick วิธีนี้แถมไม่ได้ 100% และอาจมีความคลาดเคลื่อนเกิดขึ้นได้ คะแนนสุดท้ายที่จะแสดงจึงต้องมาจาก server เพื่อความแม่นยำยิ่งขึ้น

- Frame Rate ไม่เท่ากัน (30/60/120 FPS) — รับมืออย่างไร : จากที่อธิบายไปแล้วในเรื่องการใช้ fixed timestep และ fixed update ดังนั้น ในเครื่องที่ framerate ต่ำ บางเฟรม fixed update อาจถูกเรียกหลายรอบ ส่วนเครื่องที่ frame rate สูง บางเฟรม fixed update อาจไม่ถูกเรียกเลย ทำให้ จำนวนและขนาด tick เท่ากัน ไม่ว่าจะ framerate เท่าไร
- ขอบเขต: อะไรต้อง Deterministic อะไรไม่จำเป็น

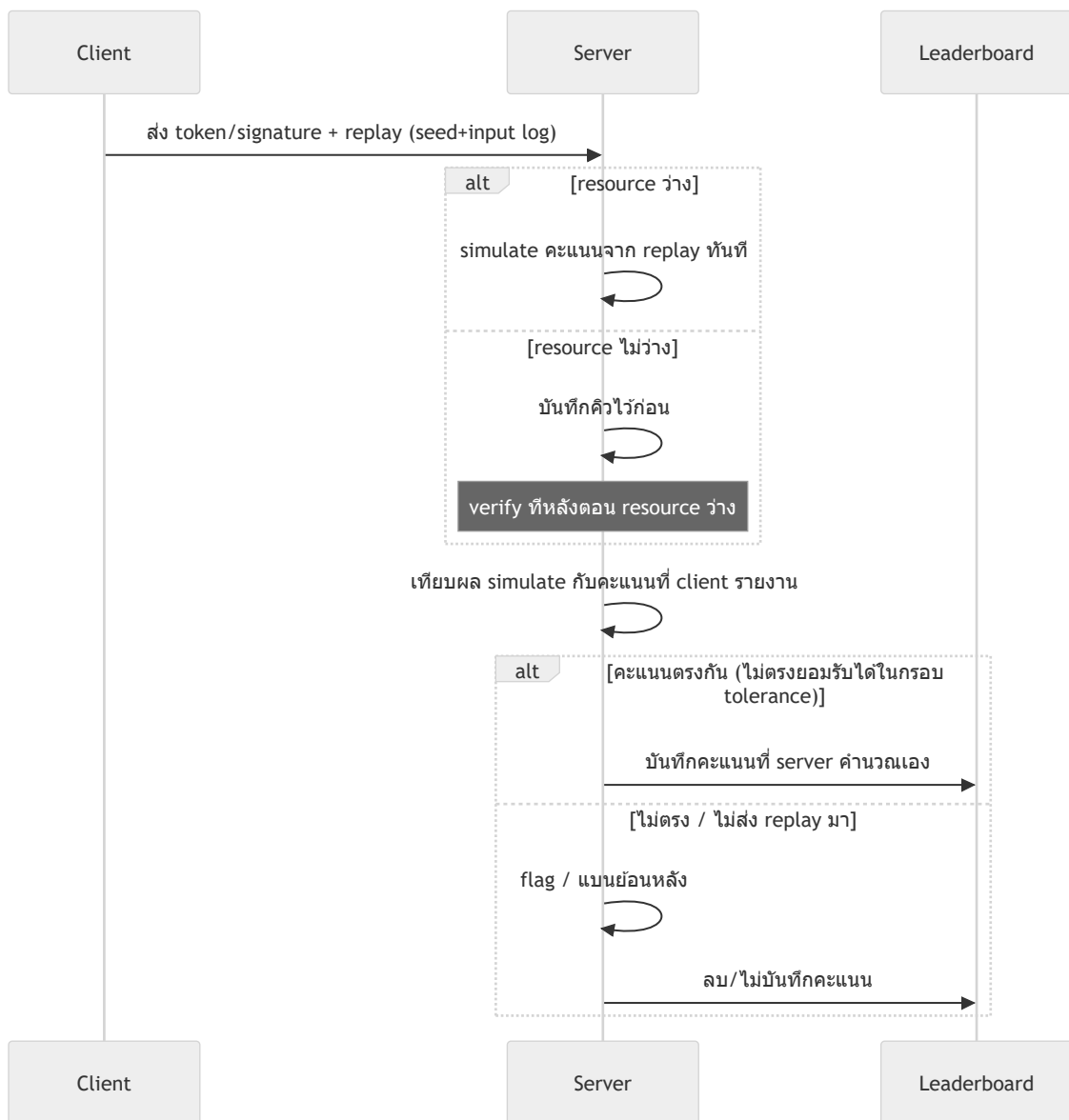
จำเป็นต้อง Deterministic	ไม่จำเป็น
speed, การสุ่ม obstacle, คะแนน	ui, เสียง

- Simulation ควรอยู่ในรูปแบบใดจึงรันได้ทั้ง Client และ Server : fixed timestep โดย simulate คะแนนออกมาโดยใช้ loop จาก replay input ที่บันทึกไว้ หากต้องการแค่คะแนน แต่หากต้องการเกม replay ฟัง client เราสามารถใช้ update ของ unity สะสม delta time ไปเรื่อยพอลถึง fixed timestep ก็รัน 1 tick โดย simulate logic จะใช้ script class และ method เดียวกัน หรือ อย่างน้อย อาจไม่ใช่ไฟล์เดียวกัน แต่ต้องเขียน logic ให้เหมือนกัน กรณี server ไม่ได้ใช้ c#



1.2 Anti-Cheat & Score Validation

- วิธีที่ทำให้ Server เชื่อถือคะแนนได้ + กันโกงแบบไหนได้/ไม่ได้ : client ต้องส่ง token หรือ signature ระบุตัวตนมาด้วย แต่ server ไม่เชื่อคะแนนจากฝั่ง client อยู่แล้วครับ server จะ simulate คะแนนเองจาก replay input ที่ client ต้องส่งมา ถ้าไม่ส่งมาก็เชื่อได้เลยว่าไม่โปร่งใส สิ่งที่ได้คือหาก มีการส่ง คะแนนที่ไม่ตรงกับที่ควรจะได้ขึ้นมาก็ไม่สามารถบันทึกได้ แต่จะกันไม่ได้หากสามารถทำสร้าง replay ปลอมขึ้นมาได้
- ต้นทุนของแนวทางที่เลือก (ข้อมูลที่ส่ง, CPU ฝั่ง Server, ค่าใช้จ่ายที่โตตามผู้เล่น) : คิดว่าไม่น่าเยอะ เพราะเราส่งกันเป็น text json อาจจะติดตรงเวลาที่ ใช้ ประมวลผลคะแนน
- แนวทางคุมค่าที่ระดับแสนคน/วัน — ตรวจสอบทุกคนหรือไม่ ถ้าไม่ เลือกตรวจใครด้วยเกณฑ์ใด : ตรวจทันทีเมื่อ resource ว่างอยู่ ถ้า resource เต็มก็ให้บันทึกไว้ แล้วให้ server รันตรวจทีละคนใน resource ที่ว่างอยู่ในอนาคต แล้วค่อยแบนหรือลบข้อมูลย้อนหลัง เกมจะได้ไหลลื่นไม่เกิดคอขวด
- เน็ตหลุดกลางเกม — ไม่ทำร้ายผู้เล่นสุจริต แต่ไม่เปิดช่องโกง : ถ้ายังอยู่ใน session เดิมแล้วเน็ตสามารถกลับมาต่อได้ ก็ไม่มีปัญหาครับ สามารถ นับถอยหลัก แล้วปล่อยให้เล่นต่อได้เลยแต่จะไม่ยอมให้เข้าเล่นผ่าน session ใหม่เช่น ปิดเกมแล้วเปิดใหม่



1.3 Scope & Risk

- ความเสี่ยงหลักอย่างน้อย 3 ข้อ + วิธีรับมือ

ความเสี่ยง	วิธีรับมือ
คะแนนที่เกิดจากการโกงส่งข้อมูล อาจขึ้นอยู่ใน leaderboard สักพัก	เริ่มเช็คจาก record อันดับต้นๆก่อนเพื่อคัดกรอง top player ที่โชว์ใน leaderboard ก่อน
หากมีการส่งข้อมูลโกงที่เหมือนกับ record จริงทุกอย่างก็อาจจะจับไม่ได้	ยอมรับความเสี่ยงนี้เพราะคิดว่าไม่คุ้มค่าที่ต้องลงความคิดลงแรงกับจุดนี้
การ resume เกมหลังเน็ตหลุดอาจทำให้เกิดบัค ซึ่งต้องแก้หลายจุด	เพิ่มการ handle ที่จะเฝ้าติดตามซึ่งเชื่อมโยงกับหลายระบบเช่น ui , gameplay

- เวลาลดจาก 3 เดือนเหลือ 6 สัปดาห์ — ตัดอะไรก่อน อะไรตัดไม่ได้เด็ดขาด เพราะอะไร

สิ่งที่ตัดได้ก่อนเพื่อให้พอกับเวลา

ตัดอะไร	เพราะอะไร
การ simulate replay ฟัง client	สิ่งที่ต้องการคือ record ไม่ใช่ replay
การ round การคำนวณค่า float	สุดท้ายก็ใช้ server คำนวนและเชื่อ server
ยอมให้เกิดคอขวดระหว่าง verify record	สิ่งที่เกิดขึ้นอาจเป็นแค่ความล่าช้า ไม่ได้ส่งผลอะไรมาก

สิ่งที่ตัดไม่ได้

Deterministic ทั้งหมด และระบบป้องกันการโกง เพราะเป็น requirement หลัก

- จุดที่เลือกทำง่ายกว่าโดยตั้งใจ ทั้งที่มีทาง "ถูกต้องกว่า" ในทางทฤษฎี พร้อมเหตุผล

เลือกทำ (ง่ายกว่า)	ทางที่ "ถูกต้องกว่า" (ไม่เลือก)	เหตุผล
round ค่าที่คำนวณได้ทุก tick + ให้ server บันทึกคะแนนที่ส่งมาไปก่อน แล้วค่อยปรับเป็นคะแนนจริงหลังตรวจสอบ	fixed point — แปลงทุกค่าที่คำนวณเป็น int แล้วค่อยเอามาคำนวน	fixed point ยุ่งยากกว่า เพราะต้องแปลงค่าของ gameplay ทั้งระบบและค่าใน Unity เป็น int และต้องมีจุดที่แปลงค่ากลับ สิ่งที่ต้องการจริงๆคือค่าที่เหมือนในทุกเครื่อง ไม่จำเป็นต้องแม่นยำสมบูรณ์แบบ
resume ใน session เดิม	resume ข้าม session	การ resume ข้าม session อาจจะดูยุติธรรมกว่าสำหรับผู้เล่นที่หลุด แต่ฝั่งผู้เล่นที่โกงจะกลายเป็นเพิ่มความเสี่ยงให้ตัวเกม หากผู้เล่นแก้ไฟล์ให้ตัวเองเหมือนกับคนที่หลุด

AI-usage disclosure

1.1 Deterministic Simulation — ผมเขียน draft เองก่อน แล้ว AI ช่วยอธิบาย concept ที่ผมไม่แน่ใจ + จัดที่ตอบไม่ครบ ผมเอาความเข้าใจไปเขียนคำตอบเองทั้งหมด สรุปที่ AI ช่วย:

- ชี้ว่า $\Delta t * speed$ แก่แค่การแสดงผล ไม่ใช่ตัวทำให้ deterministic จริง ต้องใช้ fixed-timestep แทน และ sim logic ต้องแยกเป็น plain C# ไม่ผูก MonoBehaviour (server ไม่มี Unity Player)
- อธิบาย float precision ต่างเครื่องคืออะไร ทำไมโจทย์นี้แค่เป็นพิเศษ (ใช้ replay-verification เป็น anti-cheat) และวิธีรับมือ (round ทุก tick ลด drift ได้แต่ไม่ 100%, fixed-point คือทางที่การันตีได้แต่ต้นทุนสูง)
- แก้ความเข้าใจผิดของผม: เคยคิดว่า "round แล้วเชื่อ client score" พอ — AI ชี้ว่าขัดกับข้อไหน client-authoritative score ที่เจอเองใน Part 3 server ต้อง replay คำนวนเองเสมอ
- Review จุดที่ตอบบางไป (Frame Rate ข้อ 3, ความสม่ำเสมอของ list บ้างข้อ 2) ให้ผมเติมเอง
- ข้อ 5 อธิบายว่า client ไม่จำเป็นต้องพึ่ง FixedUpdate ตรงๆ ใช้ accumulator เองใน Update() ได้ และช่วยแยกจุดประสงค์ server (เอาแค่คะแนน) vs client (ต้องเล่นได้ real-time ด้วย) ที่ผมสับสน — ส่วนประเด็น "ถ้า server ไม่ใช่ C#" เป็นของผมเองที่ต่อยอด

เนื้อหา/ประโยคทั้งหมดในคำตอบเป็นของผมเขียนเอง AI ไม่ได้เขียนคำตอบแทน มีบทบาทแค่อธิบาย concept และจัดที่ตอบไม่ครบ (AI จัดตาราง (รวมตารางขอบเขต) + วาด diagram สถาปัตยกรรม sim class ให้จากที่ผมเขียนไว้แล้ว ไม่ได้เปลี่ยนเนื้อหา)

1.2 Anti-Cheat & Score Validation — ผมเขียน draft เอง AI ช่วย review จัดไม่ครบ/ขัดแย้งกันเอง ผมแก้เองทั้งหมด:

- ข้อ 1 ตอบแค่ "กันไม่ได้" ด้าน AI ชี้ว่าลืมบอกว่า "กันได้" อะไรด้วย ผมเลยเติมส่วนกันคะแนนปลอมได้
- ข้อ 3 ("ตรวจทุกคน") ขัดกับข้อ 2 ที่ผมเองพูดว่า CPU อาจเป็นคอขวด — AI ชี้ความขัดแย้งนี้ ผมแก้เป็นแนวทาง verify async แทน

- ข้อ 4 (เน็ตหลุด) AI ชี้ว่าคำตอบตอนแรก (ไม่บันทึก ต้องเล่นใหม่) ขัดกับโจทย์เองที่บอก "ไม่ทำร้ายผู้เล่นสุจริต" ผมเลือกคงคำตอบนั้นไว้ก่อนโดยระบุว่าเป็นจุดที่ต้องให้ฝ่ายดี/PM ตัดสินใจ ต่อมาได้ second opinion จาก Fable (ผ่านทีม AI อีกตัว) ชี้ว่า "ให้ design/PM ตัดสินใจ" ไม่ใช่คำตอบที่ยอมรับได้กับคำถามที่โจทย์ถามให้ผมตอบในฐานะวิศวกรเอง ผมเลยกลับมาคิดคำตอบของตัวเองใหม่และเขียนเองทั้งหมด (resume ได้ใน session เดิมที่หลุด แต่ไม่ให้ resume ข้าม session เพื่อกันโกง)

เนื้อหา/ประโยคทั้งหมดเป็นของผมเขียนเอง AI มีบทบาทแค่ชี้จุดขัดแย้ง/ไม่ครบ ไม่ได้เขียนคำตอบแทน (AI วาด sequence diagram สรุป flow จากที่ผมเขียนไว้แล้ว ไม่ได้เพิ่มเนื้อหา/ตัดสินใจ design ใหม่)

1.3 Scope & Risk — ผมเขียน draft เอง AI ช่วย review จุดที่ตอบไม่ครบ ผมแก้เองทั้งหมด:

- ข้อความเสี่ยง 2 ข้อหลัง (replay ปลอมจับไม่ได้, เน็ตหลุดตัดจบ) ตอนแรกไม่มีวิธีรับมือเลย AI ชี้ให้เดิม ผมเดิมเอง — ต่อมาหลังแก้คำตอบเน็ตหลุดใน 1.2 ใหม่ (resume ใน session เดิม) ความเสี่ยงข้อนี้ไม่ตรงกับดี/ไซน์ใหม่แล้ว ผมเปลี่ยนเป็นความเสี่ยงเรื่องการ resume เองอาจเกิดบัค (ต้อง handle เชื่อมหลายระบบ ui/gameplay) แทน เพื่อให้ครบ 3 ข้อตามโจทย์
- เวลาลด 3 เดือนเหลือ 6 สัปดาห์: AI ชี้ว่าข้อ "ตัด replay ฟัง client" กำกวมไม่รู้หมายถึงอะไร ผมชี้แจงเอง และ AI ชี้ว่าขาดฝั่ง "ตัดไม่ได้เด็ดขาด" ไปทั้งฝั่ง ผมเดิมเอง
- ข้อสุดท้าย (จุดที่เลือกง่ายกว่าโดยตั้งใจ) ผมถาม AI เรื่อง fixed-point math หลายรอบเพราะไม่เข้าใจ: คืออะไร, ทำไม $\times 1000$ ถึงการันตี deterministic ได้ (ต่างจาก float ตรงไหน), และวิธีทำ growth แบบ % โดยไม่หลุดกลับไปเป็น float (คูณ-หารด้วย scaled-integer) — ผมเคยเข้าใจผิดว่า fixed-point "ท้าทาย" ในแง่เทคนิคเดียวๆ AI ช่วยปรับมุมมองว่าต้นทุนจริงคือต้องทำทั่วทั้งระบบ + จุดต่อกับ Unity (float-native) ไม่ใช่ตัวเทคนิคเอง ผมเอาความเข้าใจนี้ไปเขียนเหตุผลเปรียบเทียบ round vs fixed-point เอง
- AI จัดรายการความเสี่ยง, ตัดอะไรตอนเวลาลด, และจุดที่เลือกง่ายกว่า ให้เป็นตาราง จากที่ผมเขียนไว้แล้วทั้งหมด ไม่ได้เปลี่ยนเนื้อหา

เนื้อหา/ประโยคทั้งหมดเป็นของผมเขียนเอง AI ไม่ได้เขียนคำตอบแทน มีบทบาทแค่อธิบาย concept ที่ถามและชี้จุดที่ตอบไม่ครบ

ส่วนที่ 2 — Prototype

เลือก ตัวเลือก A — Deterministic Core + Replay Verification

ทำไมเลือก A

- อะไรคือความเสี่ยงที่สุดใน Design (เทียบกับตัวเลือกอื่น) ทำไมถึงต้องพิสูจน์ข้อนี้ก่อน : เนื่องจากส่วนนี้เป็น part หลักและเป็นจุดเริ่มต้นของทุกส่วนถ้าพลาดตั้งแต่ตรงนี้ถึงส่วนอื่นจะทำได้ดีแค่ไหนก็ไม่มีประโยชน์

1. แยก Simulation ออกจาก MonoBehaviour ให้รันได้โดยไม่ต้องพึ่ง Unity Player

- สถาปัตยกรรมที่ใช้ (sim class, accumulator, driver ฟังก์ชัน client/mock-server) : ใช้ menu item run plain c# โดยทั้งหมดไม่ได้ผูกกับ MonoBehaviour โดย menu item จะเป็นคน spawn ตัว simulator และทำการ start update และ dispose ตาม parameter ที่ได้ไป โดยแบ่งออกเป็น 2 mode record , replay และ ตัวคะแนนและ speed จะถูกคำนวณผ่าน updater ซึ่งอยู่ภายในตัว simulator
- การตีความ "รันได้โดยไม่ต้องพึ่ง Unity Player" ที่เลือกใช้ + เหตุผล : simulation สามารถใช้ได้โดยไม่ต้องกดรัน แต่ยังมีจุดที่ใช้ unity engine เช่น Debug Log , JsonUtility , ReplayFileIO ซึ่ง ส่วนนี้สามารถปรับเปลี่ยนได้ตาม environment ที่ใช้รัน หลักๆต้องการใช้ logic

2. Replay เก็บเฉพาะ Seed และลำดับ Input เท่านั้น

- Seed เป็นค่าที่ fixed ไว้สำหรับการ simulate ในสถานการณ์จริงค่านี้ต้องได้จาก server
- Input เก็บเป็น List เพื่อให้สะดวกต่อการเพิ่ม record

3. พิสูจน์ว่า Replay เดิมให้ผลลัพธ์ตรงกันทุกครั้ง แม้ Frame Rate ต่างกัน

- วิธีทดสอบ: รอบแรกเลือก menu Simulator/Record เพื่อเก็บ Replay ไว้ แล้วก็ลอง Replay ตามลำดับ 30 60 120

ผลลัพธ์จริง (record 1 ครั้ง ที่ 60fps แล้ว replay ตัวเดียวกันที่ 30/60/120fps):

รอบ	ReplayTime (ms)	Score	Speed	Obstacles (checksum)
Record (60fps)	0.0810	175	51.1	274
Replay 30fps	0.3062	175	51.1	274
Replay 60fps	0.0576	175	51.1	274
Replay 120fps	0.0353	175	51.1	274

- สรุปผล: จากการทดสอบทั้ง 4 รอบเห็นได้ว่าผลลัพธ์มีค่าเท่ากัน ในทุก framerate

4. ระบบตรวจสอบฝั่ง Server จำลอง

- รับ Replay แล้วยืนยันคะแนนได้ยังไง: รัน Record 1 รอบ และส่ง score และ replay ไปเข้า simulation อีกครั้งเพื่อให้ได้คะแนนใหม่ออกมา เปรียบเทียบกัน

ผลลัพธ์จริง (Client บันทึก+อ้าง score เอง → ส่ง replay ไป mock server → server re-simulate เอง เทียบกับ score ที่ client อ้าง):

ฝั่ง	ReplayTime (ms)	Score	Speed	Obstacles (checksum)
Client (Record)	0.0560	176	51.1	274
Server (Verify)	0.0463	176	51.1	274

Submit Score : 176, Verify Score : 176 — ตรงกัน server เชื่อคะแนนนี้ได้

- เวลาที่ใช้ตรวจต่อ 1 Replay: ~0.048 ms (หมายเหตุ: วัดจาก Editor ยังไม่มี network/parse overhead จริงของ production)

AI-usage disclosure

Core logic ทั้งหมด (RunnerSimulation , accumulator, Replay , RunnerSimulationWindow) ผมเขียนเอง AI มีบทบาท 2 อย่าง:

อธิบาย concept ที่ถาม — fixed tick คืออะไรและทำไมใช้ 1/60, accumulator ทำไมต้องเก็บเศษที่เหลือไว้ (carry-over) ไม่ทิ้ง, spiral of death คืออะไรและวิธีป้องกัน (cap จำนวน tick catch-up ต่อเฟรม), ทำไม Tick() ต้องไม่เป็น async, ความต่างระหว่าง record mode (input sample ต่อ outer call เข้าได้ในหลาย catch-up tick) กับ replay mode (input ต้อง index ตาม tick ไม่ใช่ตาม frame) — ทุกจุดผมตามด้วยการ trace/เขียนโค้ดเองแล้วเอา AI ช่วยเช็คความถูกมัย

Review หา bug/gap — AI ชี้ 3 รอบว่า outer loop ผูก tick count เข้ากับ simulationRate แทนที่จะผูกกับ fps (ทำให้เวลารวมที่จำลองผิด), deltaTime ที่ส่งเข้า constructor คำนวณจาก fps แทนที่จะเป็นค่าคงที่จริง, และ RunnerSimulation ไม่มีทาง "เล่นซ้ำ" จาก Replay ที่มีอยู่แล้ว (มีแต่โหมดบันทึกใหม่) ผมแก้เองทุกจุด

AI เขียนโค้ดให้ (boilerplate เท่านั้น) — ReplayFileIO.cs (Save / Load ผ่าน JsonUtility) และเพิ่ม [System.Serializable] ให้ Replay struct เพราะเป็นแค่ I/O plumbing ไม่ใช่ core sim logic — ตัดสินใจเลือก JSON แทน ScriptableObject เองก่อน (เพราะ ScriptableObject โหลดไม่ได้ถ้าไม่มี Unity Player ชัดกับ requirement ของ option นี้โดยตรง) แล้วให้ AI ช่วยเขียนโค้ด I/O ตามที่ตัดสินใจไว้

รอบ ServerSimulator + bug เพิ่มเติม — ผมเสนอเองว่าอยากให้ script แยกจำลอง "ส่ง request" (claimed score + replay) ไม่เทียบกับคะแนนที่ server คำนวณเอง ตรงกับ design ใน Part 1.2 ผมออกแบบ/เขียน ServerSimulatorEditor.cs เอง ระหว่างนั้น AI ชี้ 2 จุด: (1) ต้อง expose Score เป็น public getter ก่อนถึงจะอ่านค่าจากภายนอกได้ (2) bug ร้ายแรงจาก logic เงื่อนไขไขกลับด้านใน constructor ของ RunnerSimulation (SimMode.Replay) ที่ทำให้ path เดิม (RunnerSimulationEditor) พังด้วย NullReferenceException — ผมแก้เอง นอกจากนี้ AI แนะนำเพิ่ม obstacle checksum (เพื่อพิสูจน์ความ deterministic เน้นกว่าแค่ดู score) และแก้ปัญหา Stopwatch.ElapsedMilliseconds บิดเป็น 0 (ใช้ Elapsed.TotalMilliseconds แทน) ซึ่งผมนำไปเขียนเอง

รอบตรวจสอบสุดท้าย (Fable second opinion อีกรอบ) — ให้ Claude ขอ Fable ตรวจทั้ง 5 part พร้อมกันอีกครั้งก่อนส่งจริง เจอว่าเอกสาร Part 1.1/Part 5 อ้างว่า "round float ทุก tick" เป็นทางแก้ float drift แต่โค้ด BaseUpdater.cs ตอนนั้นไม่มีการ round จริงเลย (พิสูจน์แค่ fixed-timestep เฉยๆ) — ผมเลือกแก้โค้ดจริงแทนแก้แค่คำในเอกสาร เพิ่ม Math.Round ใน speed เอง รอบแรกที่เขียนดัน round ผิดจุด (round ค่าคงที่ที่บวกเพิ่มแต่ละ tick แทนที่จะ round ตัว speed ที่สะสมไว้ ซึ่งไม่มีผลอะไรเพราะค่าคงที่ที่ไม่มีทาง drift อยู่แล้ว) AI ชี้จุดนี้ ผมแก้เป็น round ตัว speed หลังบวกเสร็จแทน แล้วรัน demo ใหม่ทั้งหมด ผลยังตรงกันทุก framerate เหมือนเดิม (อัปเดตตัวเลขในตารางข้างบนเป็นชุดใหม่จากรอบนี้)

เนื้อหา/สถาปัตยกรรม/การตัดสินใจออกแบบทั้งหมดเป็นของผมเอง AI ไม่ได้ออกแบบหรือเขียน core logic แทน มีบทบาทแค่ อธิบาย concept, ชี้ bug, และเขียน boilerplate ตามที่สั่ง

ส่วนที่ 3 — Code Review

1. Review ตามที่จะเขียนจริงใน Pull Request

- แก้ compile error ด้วยครับ
- แก้เรื่อง Determinism (random,update,deltatime)
- ปรับ Code style ไม่ตรงกับทีม (serializefield , encapsulate , ghost number)
- ปรับ การทำงานกระจุกตัวอยู่ในโค้ดเดียวกันไป (srp)
- ใช้ unitask แทน IEnumerator และ ใช้ \$" " แทน "" + ""
- แก้ spawn object ไม่ถูก destroy
- แก้ เกม infinity loop ไม่มีวันจบ
- Client ส่ง score ไปตรงๆ ไม่มี token และ ไม่มีการ verify replay record — อันนี้ต้องคุยกับทีม backend...
- เพิ่ม error handler ใน request
- singleton ไม่ได้เช็ค Instance Null? ทำให้มี Instance ใหม่โดยที่ instance เก่ายังอยู่
- เข้ารหัส player pref
- ไม่แน่ใจว่าทำไมเรียก CheckCollision เอง ทำไมไม่ใช้ observer กับ api ของ unity (oncollisionenter)

2. จัดกลุ่มประเด็น

ต้องแก้ก่อน Merge

1 Compile error

- ไม่ได้ใส่ using UnityEngine.UI(text) และ using System.Collections.IEnumerator)
- ไม่มี function SpawnObstacle กับ CheckCollision
- ปกติผมไม่ได้เช็ค compile error ตอนตรวจ pr อันนี้ผมเอาโค้ดมาลอง compile อาจจะ โกงไปหน่อย

2 Code style

- ทุก global variable เป็น public ทั้งหมด ไม่สามารถบอกได้ว่าตัวไหนต้องการจะเป็น serializefield หรือ ไม่เป็น
- ทุก global variable ไม่มีความจำเป็นต้องเป็น public เลย ยกเว้น Instance ส่วนตัวอื่นๆสามารถใช้ [serializefield] private แทนได้ ถ้าอยากให้เห็นใน Inspector
- Ghost number เยอะมากต้องประกาศเป็น private const ไม่ก็ แยกไฟล์ config.cs หรือ const.cs ไปเลย
- โค้ดนี้เป็นโค้ด gamemanager ควรเป็นหน่วยความคม feature ใหญ่อื่นๆ ผิดหลัก SRP ซึ่งโค้ดนี้สามารถกระจายไปยังระบบอื่นๆได้ดังนี้ ระบบคำนวณความเร็ว,ระบบคำนวณคะแนน,ระบบ spawn object,ระบบsound, ระบบ localsave , ระบบsendrequest
- ทุก function ต้องใส่ encapsulate private public

3. Performance

- ห้ามใช้ IEnumerator ใช้ unitask เท่านั้น ในโค้ดตอนนี้ไม่สามารถ cancel ได้ ไม่รู้เลยว่าทำเสร็จเมื่อไร
- ห้ามใช้ "text" + "text" การทำแบบนี้จะสร้าง gc แคมอยู่ใน update ด้วย ใช้ \$"{text}" หรือ string.format หรือ Zstring

4 Risk

- ต้อง random จาก seed ให้ตรงกับ requirement

- การใช้ update กับ time.deltatime ช่วยแค่การแสดงผลให้ดูเสมือน เท่านั้นคะแนนและความเร็วที่คำนวณออกมาไม่ได้ Determinism
- มีแค่การ spawn obstacle ไม่มีการ destrop / release / return
- เมื่อ gameover ไม่มีการ return ออกจาก loop สามารถเกิดการรบกวน gameover ซ้ำๆ เกิดการยิงรีเคสซ้ำๆ แล้วเกมก็ไม่จบ update ยังทำต่อ เพราะไม่มี flag ตัวไหนบอกว่าเกมรันอยู่รึเปล่า
- ไม่มี error handler หรือ เช็ค status request ที่ได้มา
- Client ส่ง id และ score ไปตรงๆ ไม่มี token/signature และ ไม่มี replay record อันนี้ต้องคุยกับทีม backend แก่ server ให้รับ token/signature และ replay record ไปด้วย

ควรแก้แต่รอได้

1. Singleton ไม่มีการเช็คว่ามี instance เดิมอยู่รึเปล่า อาจทำให้มี object ค้างใน scene
2. key playerpref ควรเข้ารหัส ไม่ควรใช้ key text ตรงๆ

ข้อสังเกต

1. Unity มี oncollisionenter ontriggerenter อยู่แล้วทำไมต้องมาเรียกเช็ค CheckCollision() เอง ใช้เป็น observer ไปเลยไม่ได้หรือ

3. ประเด็นร้ายแรงที่สุด + วิธีแก้

เนื่องจากเป็นเกมที่มีผู้เล่นจำนวนมากการที่เกมบัค หรือ performance ไม่ดี ยังแย่น้อยกว่าการมีการแฮคคะแนน หรือ ส่งคะแนนผิด ประเด็นที่ควรแก้ที่สุดคือ score submission ตามที่เห็นการส่ง request จากใน code แสดงให้เห็นว่าเซิร์ฟเวอร์ไม่ได้รับ payload token/signature , replay record ทำให้ server ไม่สามารถตรวจสอบคะแนนได้ token/signature เป็นเพียงเกราะชั้นแรก แต่ replay record เป็นตัวที่ใช้เช็คจริงๆ ต้องไปคุยกับฝั่งคนเขียน server ให้ได้รับ token/signature และ verify score โดยใช้ replay record ด้วย

4. ถ้าเป็น Prototype ต้องส่งภายในวันพรุ่งนี้ — Review จะเปลี่ยนไปยังไง

ทุกอย่างเหมือนเดิมยกเว้น

- ปรับ การทำงานกระจุกตัวอยู่ในโค้ดเดียวเกินไป (srp)
- code style ทั้งหมด

เนื่องจากผมมองว่าเวลาทั้งหมดนี้ควรทำเสร็จในเวลาไม่กี่ชั่วโมงยกเว้นการปรับโครงสร้างโค้ดซึ่งอาจจะใหญ่ไป ถ้ามีโอกาสเกิดบัคเพิ่มมากกว่าเดิม หรือถ้าไม่ทันจริงๆ การไม่ปรับไปใช้ unitask ก็สามารถละไว้ก่อนได้

AI-usage disclosure

ผมไล่หาบัค/ปัญหาในโค้ดด้วยตัวเองก่อนทุกข้อ แล้วให้ Claude (AI assistant) ช่วยตรวจทานเพิ่มเติมเป็นรอบๆ ไป ตามรายละเอียดนี้:

ข้อ 2 (จัดกลุ่มประเด็น) — ผมเขียน draft แรกเองก่อน (compile error, code style, SRP, UniTask/GC) แล้ว AI ช่วยตรวจทานและชี้จุดที่ผมพลาดไปทั้งหมด 4 จุด แบ่งเป็น 2 รอบ:

- รอบแรก: (1) ช่องโหว่ client-authoritative score submission (ส่ง score ผ่าน GET ไม่มี token/signature) และ (2) ความเชื่อมโยงระหว่าง Random.value ที่ไม่ seed กับ requirement เรื่อง Determinism ในส่วนที่ 1
- รอบสอง: (3) ไม่มี state guard กัน GameOver() ถูกเรียกซ้ำ (ทำให้ยิง request ซ้ำและเกมไม่จบ) และ (4) obstacles list ไม่มีการลบ object ที่ถูก destroy แล้ว เสี่ยง MissingReferenceException

ผมนำทั้ง 4 จุดนี้ไปเขียนเพิ่มเข้า list เองทั้งหมด (คำอธิบาย/เหตุผลเป็นของผม)

ข้อ 1 (PR-style comment) — ผมเขียนตามสไตล์ที่ใช้จริงในงาน (แบบหัวขั เป็น bullet) แล้ว AI ช่วยเทียบกับ list ในข้อ 2 ขัว่าข้อ 1 ที่ผมเขียนไว้ตอนแรกขาด blocker ไป 2 จุด คือ compile error และเรื่อง token/signature — ผมเพิ่มเข้าไปเอง ส่วนสไตล์การเขียน (หัวขั ไม่มี greeting) ผมยืนยันว่าเป็นของผมเอง ไม่ได้ปรับตาม AI

ข้อ 3 (ประเด็นร้ายแรงสุด) — ผมเลือกประเด็นและเขียนคำอธิบาย fix แรก (เพิ่ม token/signature ผัง backend) เองทั้งหมด แล้ว AI อธิบายเพิ่มเติม (ผมถามเอง) เรื่อง:

- ความแตกต่างระหว่าง auth token ตอน login กับ signature ต่อ request (คนละปัญหาขั — token ยืนยันตัวตน ไม่ได้ยืนยันว่าค่า score เป็นความจริง)
- ข้อจำกัดของ signature-based fix (client secret ถูก extract ได้ในทางทฤษฎี) และทางเลือกที่แน่นกว่า (server re-simulate จาก seed+replay แทนการเชื่อ client)

ผมนำแนวคิดนี้มาเขียนเป็นย่อหน้าเสริมในคำตอบเอง (ประโยคเป็นของผม รวมถึงส่วนที่เพิ่มเองเรื่อง risk การ simulate คลาดเคลื่อน/ต้องปรับจูน ซึ่ง AI ไม่ได้พูดถึง)

ข้อ 4 (ถ้าต้องส่งพรุ่งนี้) — ผมเขียนคำตอบเองว่าจะเลื่อน SRP refactor ออกไปก่อน พร้อมเหตุผล แล้ว AI เสนอเพิ่มว่า UniTask migration ก็น่าจะเข้าเกณฑ์เดียวกัน (effort/risk สูง) ควรพิจารณาเลื่อนได้เหมือนกัน — ผมรับมาแต่ปรับเป็นเงื่อนไข ("ถ้าไม่ทันจริงๆ") แทนที่จะเลื่อนแบบเด็ดขาด เพราะยังอยากทำถ้ามีเวลาพอ

ตลอดทั้ง 4 ข้อ **AI ไม่ได้เขียนคำตอบแทนผม** — บทบาทคือช่วยตรวจทานหา blind spot ที่ผมมองข้าม และอธิบาย concept ที่ผมถามเพิ่มเมื่อไม่แน่ใจ (เช่น determinism, token vs signature) เนื้อหา/เหตุผล/ประโยคทั้งหมดในคำตอบเป็นของผมเขียนเอง

การแก้ไขรอบสอง (หลังเขียน Part 1 เสร็จ) — ผมให้ Claude ขอ second opinion จาก Fable (AI อีกตัว) ตรวจ Part 1 กับ Part 3 พร้อมกัน เพื่อเช็คว่าสองส่วนขัดแย้งกันเองมั้ย Fable เจอ 3 จุดที่ผมแก้เองทั้งหมด:

- ข้อ 2: Random.value ไม่ seed เดิมจัดไว้ใน "ข้อสังเกต" (ระดับต่ำสุด) แต่ขัดกับ Part 1 ที่บอกว่า seed คือ requirement หลักของทั้งโหมด หัวขัน่าจะพ่วงถ้าไม่มี ผมย้ายขัไป "ต้องแก้ก่อน Merge" เอง
- ข้อ 1, 2: โค้ดใช้ Update() + Time.deltaTime (variable timestep) ซึ่งเป็นปัจจัยทำลาย Determinism ตามที่เขียนไว้เองใน Part 1 แต่รีวิวดิไม่เคยพูดถึงเลย ผมเพิ่มเข้าไปทั้งในข้อ 1 (PR comment) และข้อ 2 (Risk)
- ข้อ 3: คำอธิบาย fix เดิมเสนอ token/signature กับ server-replay เป็นทางเลือกที่เทียบเท่ากัน ("หรือ อีกทางหนึ่ง") ทั้งที่ token ยืนยันแค่ตัวตน ไม่ได้ยืนยันว่าค่า score จริง ผมแก้ประโยคให้ชัดว่า token/signature เป็นแค่ "เกราะชั้นแรก" ส่วน replay record คือตัวที่เช็คคะแนนจริง
- ข้อ 4: เดิมเลื่อนแค่ SRP อย่างเดียวตอนโปรโตไทป์ต้องส่งพรุ่งนี้ Fable ขัว่า code style อื่นๆ (ghost number, encapsulate) ก็เป็น risk-to-correctness ต่ำเหมือนกัน น่าจะเลื่อนได้ด้วย ผมเพิ่ม "code style ทั้งหมด" เข้าไปในรายการที่เลื่อนได้เอง

เนื้อหา/ประโยคที่แก้ทั้งหมดเป็นของผมเขียนเอง Fable มีบทบาทแค่ชี้จุดขัดแย้งข้ามเอกสาร ไม่ได้เขียนคำตอบแทน

ส่วนที่ 4 — LiveOps Problem Solving

สถานการณ์: หลังเปิด Weekly Tournament ไปแล้ว 1 สัปดาห์

- ผู้เล่นราว 0.4% รายงานว่าคะแนนที่ได้บนเครื่องตัวเอง ไม่ตรงกับคะแนนที่ขึ้นบน Leaderboard
- อาการนี้เกิดกับเครื่อง Android ระดับล่างมากกว่ารุ่นอื่นอย่างชัดเจน
- Top 10 ของ Leaderboard มีคะแนนสูงกว่าอันดับ 11-100 อย่างผิดสังเกต
- ทีม Support เริ่มได้รับข้อความร้องเรียนเพิ่มขึ้นต่อเนื่อง
- ทีมงานเฝ้าระวังรอบถัดไปจะเริ่มในอีก 4 วัน และมีการประกาศแจกรางวัลไปแล้ว

ไม่แน่ใจว่า "มีการประกาศแจกรางวัลไปแล้ว" คือรางวัลของรอบถัดไป หรือรางวัลของรอบที่เปิดไปแล้ว ในข้อนี้จะถือเป็นรางวัลของรอบที่เปิดไปแล้ว คำตอบทั้งหมดนี้อิงจาก system design ใน part 1

1. สมมติฐานสาเหตุของแต่ละอาการ + วิธีพิสูจน์

- ระบบยังอยู่ระหว่างการ verify replay record : รอผลการ verify
- มีการโกงเกิดขึ้น โดยผู้โกงส่งข้อมูลเหมือนของจริงๆทุกประการทำให้ระบบจับไม่ได้ หรือ : เช็ค replay record log เพื่อหาพฤติกรรมที่น่าสงสัยเนื่องจาก top player คงไม่ได้เล่นเกมเดียวแล้วแต่ขึ้น leaderboard ทันทีแน่นอนต้องเกิดจากการเล่นซ้ำๆบ่อยมากๆ
- เรื่องคะแนนไม่เท่ากับ server เกิดขึ้นได้เพราะการคำนวณ float ที่ต่างกันในระดับฮาร์ดแวร์ : ทดสอบ replay จาก record ผู้เล่นที่มีปัญหา

2. ข้อมูลที่อยากได้เพิ่ม

- replay record log ของผู้เล่นอันดับ 11-100 : ข้อมูลนี้จะเก็บได้ก็ต่อเมื่อมีการ submit score ขึ้นมา
- รุ่นมือถือ android : ควรเก็บ firebase analytic เก็บข้อมูลพวกนี้ไว้ให้แล้ว

3. 3 สิ่งที่ต้องลงมือทำก่อน ภายใน 4 วันนี้

1. แจ้งผู้เล่นชะลอการแจกรางวัลจนกว่าจะมีการตรวจสอบเสร็จสิ้น : สามารถทำได้ง่ายและทำได้ทันทีทำไปก่อนได้เลย
2. ตรวจสอบคะแนน top player และจัดการ ban account ที่มีพฤติกรรมต้องสงสัย : เป็นส่วนที่ต้องใช้เวลาในการเช็คจริงๆสำคัญกว่าข้อแรกแต่ข้อแรกทำได้เลยและใช้เวลาไม่นานเลยทำข้อแรกก่อน
3. ทำ fixed point เพื่อให้คะแนนจากฝั่ง client และ server ตรงกัน : เนื่องจากกระทบผู้เล่นส่วนน้อย และ ไม่ส่งผลกระทบต่อรางวัล สามารถเก็บไว้ทำหลังสุดได้ และ ถึงไม่ทัน deadline ก็ไม่เป็นไร

4. การสื่อสารกับฝ่ายบริหารและผู้เล่น

- ผู้บริหาร เคสพวกนี้เป็นเคสที่เรารู้ล่วงหน้าแล้วว่ามีโอกาสเกิด แต่เราคิดว่าไม่คุ้มที่จ่าเลยปล่อยไว้ เนื่องจากมันไม่เกิดขึ้นได้ง่าย แต่ในเมื่อเกิดขึ้นไปแล้วก็เสนอวิธีแก้ และ cost ต้องการเพื่อให้ผู้บริหารตัดสินใจ
- ผู้เล่น ข้อมูลอยู่ระหว่างตรวจสอบสำหรับ top player ที่มีคะแนนมากกว่าปกติ ส่วนเรื่องคะแนนไม่ตรงเป็นข้อผิดพลาดของการแสดงผลเท่านั้น คะแนนที่ได้จาก server เป็นคะแนนที่เชื่อถือได้

5. ป้องกันไม่ให้อาการลักษณะนี้เกิดซ้ำ

- ทำ fixed point ที่เราเลือกที่จะไม่ทำเพราะต้องการประหยัดเวลา เพื่อแก้ปัญหาคะแนน
- เพื่อยิง session start ตอนเริ่มเกมเพื่อเก็บ timestamp ไว้ แล้วตอนจบเกมก็ยิง submit มาพร้อมกับ timestamp ตอนจบ เพื่อเทียบเวลาในเกม เน้นอนว่ายังมีความเสี่ยง แต่มันจะยากขึ้นกว่าเดิมมาก

AI-usage disclosure

ผมเขียน draft แรกเองทุกข้อก่อน แล้วให้ AI ช่วย review ชี้จุดที่ตอบไม่ครบ/ไม่สอดคล้องกัน ผมแก้ไขทั้งหมด:

- ผมถามเองว่าคำตอบ Part 4 ควรอิงจาก system design ใน Part 1 มั้ย เพราะเห็นว่าเคสพวกนี้ถูกตักไว้แล้วใน Part 1 — AI ยืนยันว่าควรอิง และชี้จุดเชื่อมที่ผมยังไม่เห็น: อาการ "คะแนนไม่ตรงบน Android ต่ำ" ตรงกับ float precision risk ที่ยอมรับไว้ใน 1.1, อาการ "Top 10 สูงผิดปกติ" ตรงกับ risk เรื่อง replay ปลอมจับไม่ได้ใน 1.3
- ข้อ 1: draft แรกมี 2 สมมติฐานปนอยู่ในบรรทัดเดียว AI ชี้ให้แยก ผมแยกเอง
- ข้อ 2: ตอบแค่ list ข้อมูลที่อยากได้ ไม่ได้ตอบว่าควรเก็บไว้ก่อนเปิดระบบมั้ย AI ชี้ให้เติม ผมเติมเอง
- ข้อ 3: draft แรกมีแค่ 1 ข้อ (ต้องมี 3) และเหตุผลจัดลำดับอ่อน AI ชี้ให้เติมและคิดเหตุผลจาก deadline/impact จริง ผมเขียนเอง — รอบแรกที่ผมเขียนเหตุผลข้อ fixed-point ผมพลาดเขียนว่า "เป็นแค่การแสดงผล" ซึ่งขัดกับเหตุผลที่ต้องแก้ด้วย fixed point (ถ้าแค่แสดงผลไม่ต้องแก้ตัวคำนวณ) AI ชี้ความขัดแย้งนี้ ผมแก้เหตุผลใหม่เอง
- ข้อ 4: ผังผู้บริหาร ผมเขียนจบด้วย "ขอความเห็นควรแก้มั้ย" AI ถามว่ารู้จักคำว่า "sign-off" มั้ย (ผมถามเอง) แล้วอธิบายว่าต่างจากการถามความเห็นตรงที่เสนอแผนมาก่อนแล้วขออนุมัติ ผมปรับคำตอบเป็นเสนอวิธีแก้+cost ให้ผู้บริหารตัดสินใจเอง
- AI แก่คำผิด (typo) หลายจุดให้ตามที่ขอ ไม่ได้เปลี่ยนเนื้อหา

เนื้อหา/ประโยคทั้งหมดเป็นของผมเขียนเอง AI ไม่ได้เขียนคำตอบแทน มีบทบาทแค่อธิบาย concept ที่ถามและชี้จุดที่ตอบไม่ครบ/ขัดแย้งกันเอง

ส่วนที่ 5 — Technical Decision Record

บริบท

- ปัญหาคืออะไร ข้อจำกัดมีอะไรบ้าง : ปัญหาคือ requirement ต้องการ ด้านชุดเดียวกันเป๊ะทุกคน และ ระบบต้องตรวจสอบคะแนนได้ โดยที่ผู้เล่น 40% เป็น android ระดับล่าง และ คุณภาพเน็ตไม่แน่นอน

ทางเลือกที่พิจารณา (อย่างน้อย 3 ทาง)

- ทางเลือกที่ 1 + ข้อดี/ข้อเสีย : ใช้ Time.deltaTime + seed ในการคำนวณตัวเลข ข้อดีคือ การแสดงผลออกมาใกล้เคียงกันในหลายๆ framerate ข้อเสียคือในระยะยาวจะเห็นข้อแตกต่างได้ชัดเจนในเครื่องที่ framerate ไม่เท่ากัน
- ทางเลือกที่ 2 + ข้อดี/ข้อเสีย : ใช้ fixed timestep + seed + round ตัวเลขทุก tick ในการคำนวณตัวเลข ข้อดีคือ การแสดงผลออกมาเท่ากันในทุกๆ framerate ข้อเสีย เนื่องจากความแตกต่างของฮาร์ดแวร์ยังมีโอกาสที่ตัวเลขคะแนนฝั่ง client และ server จะไม่เท่ากัน
- ทางเลือกที่ 3 + ข้อดี/ข้อเสีย : ใช้ fixed timestep + seed + fixed point ข้อดีคือแก้ปัญหามาจากทางเลือกที่ 2 ที่ตัวเลขอาจคำนวณออกมาไม่ตรงกัน ข้อเสีย คือต้องแก้ตัวเลขหลายจุด แล้วยังต้องเปลี่ยนกลับด้วย เพิ่มความซับซ้อนของระบบ และ เสียเวลา

สิ่งที่เลือกและเหตุผล

เลือกทางเลือกที่ 2 เพราะ การใช้ fixed deltatime + seed ช่วยแก้ปัญหการแสดงผลในแต่ละเครื่องที่ framerate ต่างกันให้ตรงกันตาม requirement แม้ว่า round ตัวเลขจะไม่ได้ perfect 100% แต่ค่าก็มีความใกล้เคียงสูง โอกาสผิดพลาดเกิดขึ้นได้น้อย และสามารถจบงานได้ไวกว่า

ผลที่ตามมา

- ผลดี ลดเวลาการทำงานที่อาจจากมากเกินไป และ ทางนี้ยังสามารถทำให้ การตรวจสอบคะแนน และ ทำ replay verify ใช้งานได้
- ผลที่ต้องยอมรับ ระบบที่ได้อาจไม่สมบูรณ์แบบ 100% มีโอกาสเกิดข้อผิดพลาดได้ แต่อยู่ในเกณฑ์ที่รับได้

เงื่อนไขที่จะทำให้ต้องกลับมาทบทวนการตัดสินใจนี้ใหม่

มีการเกิดข้อผิดพลาดบ่อยครั้ง และได้รับการอนุมัติให้ทำการแก้

AI-usage disclosure

ผมพบว่าแต่ละหัวข้อต้องกรอกอะไรตอนเริ่ม AI ช่วยชี้ว่าเนื้อหาส่วนใหญ่เอามาจากที่เขียนไว้แล้วใน Part 1/2 ได้เลย (ไม่ต้องคิดใหม่) พร้อมเสนอว่าเรื่องไหนในงานทั้งหมดน่าจะเป็น "การตัดสินใจสำคัญที่สุด" (fixed-timestep เพราะเชื่อมทุก part เป็นเส้นเดียว) — ผมเห็นด้วยและเลือกเรื่องนี้เอง ส่วนเนื้อหาจริงในแต่ละหัวข้อ (บริบท, ทางเลือก 3 ทาง, เหตุผล, ผลที่ตามมา, เงื่อนไขทบทวน) ผมเขียนเองทั้งหมดจากความเข้าใจที่สร้างมาตลอด Part 1-2

AI ชี้จุดที่ตอบไม่ครบ 1 จุด: "ผลดี" ตอนแรกเขียนแค่เรื่องประหยัดเวลา ไม่ได้พูดถึงผลดีที่สำคัญกว่า (เปิดทางให้ replay-verification ใช้งานได้จริง) ผมเติมเอง

เนื้อหา/ประโยคทั้งหมดเป็นของผมเขียนเอง AI ไม่ได้เขียนคำตอบแทน มีบทบาทแค่ช่วยจัดกรอบว่าแต่ละหัวข้อควรดึงข้อมูลจากไหน และชี้จุดที่ตอบไม่ครบ